DocuKids – Documentación técnica Entrega 6

Equipo: Lucas Areco, Paula Bigorra, Patricio Grano, Valentina Horociuk · Actualizada tras el Sprint 4 · Web: https://docukidsfrontend.vercel.app/

> Este documento está verificado contra el código de los dos repositorios (frontend y Docukidbackend) y es la referencia vigente para levantar el proyecto, configurarlo y consultar la API y el modelo de datos.

1. Qué es DocuKids

DocuKids ayuda a madres, padres y tutores a tener en un solo lugar los documentos de sus hijos (DNI, vacunas, fichas escolares, estudios médicos) y sus turnos. La app lee la fecha de vencimiento de cada documento con OCR, guarda el archivo en el Google Drive de la propia familia, crea los turnos en Google Calendar y avisa por correo antes de que algo venza.

2. Arquitectura

Pieza	Tecnología	Dónde corre
Frontend web	React Native + Expo (SDK 57), Expo Router, TypeScript, react-native-web	Vercel
Backend	Python 3.10+, FastAPI, SQLAlchemy, asyncpg	Render (plan gratuito)
Base de datos y autenticación	PostgreSQL + Supabase Auth	Supabase
Archivos	Google Drive del usuario	Google
Turnos	Google Calendar del usuario	Google
Correos de aviso	Gmail API	Google
Lectura de fechas (OCR)	Gemini Flash / Flash-Lite	Google AI Studio

Pieza	Tecnología	Dónde corre
Aviso diario de vencimientos	Llamada diaria a /scheduler/check	cron-job.org
Pruebas end-to-end	Playwright	Local

Flujo general: el frontend inicia sesión directamente contra Supabase y envía el token JWT al backend en cada pedido. El backend valida el token y es el único que habla con Drive, Calendar, Gmail y Gemini. El frontend nunca maneja tokens de Google.

3. Cómo levantar el frontend

Requisitos: Node.js 18 o superior y npm. No hace falta Android Studio para la versión web.

1. Clonar el repositorio: `git clone https://github.com/lucastomasareco/docukidsfrontend.git` y entrar a la carpeta.
2. Instalar dependencias: `npm install`
3. Crear un archivo `.env` en la raíz con las variables de la sección 4.
4. Levantar la versión web en desarrollo: `npm run web`. Por defecto queda en http://localhost:8081.
5. Generar la versión de producción: `npm run build`, que ejecuta `expo export -p web` y deja los archivos en `dist/`.

Despliegue en Vercel: el archivo `vercel.json` ya define el comando de build (`npx expo export -p web`), la carpeta de salida (`dist`) y una regla que redirige todas las rutas a `index.html`. Alcanza con conectar el repositorio y cargar las variables de entorno en el panel de Vercel.

Pruebas end-to-end: con la app levantada en el puerto 8081, ejecutar `npx playwright test`. Corre en Chromium, Firefox y WebKit. El reporte queda en `playwright-report/`.

Cómo levantar el backend

Requisitos: Python 3.10 o superior, un proyecto de Supabase, un cliente OAuth de Google de tipo Aplicación web y una API key de Gemini (Google AI Studio).

1. Entrar a la carpeta del backend, crear el entorno virtual con `python -m venv venv` y activarlo.
2. Instalar dependencias: `pip install -r requirements.txt`

3. Copiar `env.example` (el archivo se llama así, sin punto inicial) como `.env` y completar las variables de la sección 4. Si falta alguna obligatoria, el servidor no arranca y lista todas las que faltan.
4. En el Editor SQL de Supabase, crear las tablas base (`users` , `children` , `documents` , `appointments`) con el script de la Guía v1.2, sección 3. El backend no crea tablas por su cuenta. Después correr las migraciones de `supabase/migrations` en este orden: `add_folders` , `add_child_drive_folders` , `add_families` , `add_users_google_email` , `add_documents_no_expiry` , `add_documents_mime_type` , `add_documents_drive_migrated` .
5. Levantar el servidor: `uvicorn app.main:app --reload` . Queda en http://localhost:8000 y Swagger en http://localhost:8000/docs.

En producción (Render): cargar las variables en el panel de Render, no en el repositorio. `GOOGLE_REDIRECT_URI` tiene que ser idéntica, carácter por carácter, a la registrada en Google Cloud (termina en `/auth/google/callback`). `WEB_ORIGIN` debe ser la dirección de la web, https://docukidsfrontend.vercel.app, sin barra final. En cron-job.org se configura una llamada diaria `POST /scheduler/check` con la cabecera `X-Cron-Secret` .

Pendiente de confirmar: el modelo del backend usa la columna `documents.last_notified_at` (control de correos ya enviados) y no hay una migración que la cree. Si la base no la tiene, ejecutar `ALTER TABLE documents ADD COLUMN last_notified_at DATE;` .

4. Variables de entorno

Frontend (archivo .env, todas con prefijo EXPO_PUBLIC_)

Variable	Para qué sirve
EXPO_PUBLIC_API_URL	URL pública del backend en Render, o http://localhost:8000 en desarrollo
EXPO_PUBLIC_SUPABASE_URL	URL del proyecto de Supabase
EXPO_PUBLIC_SUPABASE_ANON_KEY	Clave anónima (pública) de Supabase

El archivo `.env` está en el `.gitignore` y no se sube al repositorio. Las variables con prefijo EXPO_PUBLIC_ quedan visibles en el navegador, por eso solo se usan la URL y la clave anónima, que son públicas por diseño. Si faltan, `lib/supabase.ts` usa valores de

relleno para que la app no se caiga al arrancar.

Backend (archivo .env, plantilla env.example)

Variable	Obligatoria	Para qué sirve
DATABASE_URL	Sí	Conexión a PostgreSQL en Supabase. Se escribe como postgresql://...; el backend la adapta a asyncpg
SUPABASE_JWKS_URL	Sí	Claves públicas para verificar la firma de los tokens: https://<proyecto>.supabase.co/auth/v1/.well-known/jwks.json
STATE_SECRET	Sí	Clave propia para firmar el parámetro state del OAuth de Google
CRON_SECRET	Sí	Clave que cron-job.org envía en X-Cron-Secret
GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET	Sí	Cliente OAuth de Google
GOOGLE_REDIRECT_URI	Sí	Debe coincidir exactamente con la registrada en Google Cloud
GEMINI_API_KEY	Sí	OCR con Gemini
WEB_ORIGIN	No (vacío)	Dirección de la web; sin ella Google no puede devolver a la versión web, así que en producción hay que cargarla
SUPABASE_SECRET_KEY	No (vacío)	Permite a DELETE /account borrar al usuario de Supabase Auth; sin ella esa ruta devuelve error
FRONTEND_URL	No (docukids://)	Destino de retorno para la app móvil
CORS_ORIGINS	No (*)	Orígenes permitidos, separados por coma; conviene restringirlo a la web
ENV	No (production)	Con el valor development se activa /auth/google/url-dev
GEMINI_MODEL	No (gemini-2.0-flash-lite)	Modelo de OCR; no usar modelos Pro porque son de pago
NOTIFICATION_THRESHOLD_DAYS	No (15)	Días antes del vencimiento en que se envía el correo
APP_TIMEZONE, APPOINTMENT_TIMEZONE	No (America/Argentina/Buenos_Aires)	Zona horaria para calcular «hoy» y para los eventos de Calendar

SUPABASE_JWT_SECRET ya no se usa: la validación pasó de un secreto compartido a JWKS. Las variables SUPABASE_URL y la clave pública son solo del frontend.

5. Endpoints de la API

El backend es FastAPI, así que expone Swagger automáticamente en `/docs` (agregar capturas de esa pantalla a la entrega). Todos los endpoints requieren la cabecera `Authorization: Bearer <token de Supabase>`, salvo los indicados. La lista siguiente sale de las llamadas reales que hace el frontend.

Cuenta y conexión con Google

Método	Ruta	Descripción
GET	/	Chequeo liviano; se usa para despertar al servidor gratuito
GET	/auth/google/url	Devuelve la URL de autorización de Google (recibe return_to)
GET	/auth/google/status	Indica si la cuenta ya está conectada con Google
GET	/auth/google/callback	Vuelta de Google. No lleva token; redirige a la web o a la app según WEB_ORIGIN y FRONTEND_URL
GET	/auth/google/url-dev	Solo para desarrollo: existe únicamente si ENV=development y no pide autenticación
DELETE	/account	Elimina la cuenta; el parámetro delete_drive decide si también se borran los archivos de Drive

Hijos

Método	Ruta	Descripción
GET	/children	Lista los hijos
POST	/children	Crea un hijo (nombre, fecha de nacimiento y avatar opcionales)
PUT	/children/{id}	Edita un hijo
DELETE	/children/{id}	Elimina un hijo

Documentos y carpetas

Método	Ruta	Descripción
POST	/upload	Sube un archivo (multipart, hasta 50 MB; el tipo real se detecta por su firma binaria) y lee su vencimiento con OCR
GET	/documents/{child_id}	Lista los documentos de un hijo con su estado calculado
PATCH	/documents/{id}	Edita nombre, fecha, carpeta o marca «sin vencimiento»
GET	/documents/{id}/thumbnail	Devuelve la imagen del documento: size=small (240 px) o large (1600 px). La baja el backend desde Drive
DELETE	/documents/{id}	Borra el documento; el archivo va a la papelera de Drive
GET	/folders	Lista las carpetas
POST	/folders	Crea una carpeta (también en Drive)
PATCH	/folders/{id}	Renombra una carpeta (también en Drive)
DELETE	/folders/{id}	Borra una carpeta (las 3 de base no se pueden borrar)

Papelera (Sprint 4)

Método	Ruta	Descripción
GET	/trash/documents	Lista los documentos borrados, con los días que les quedan (máximo 30)
POST	/trash/documents/{drive_file_id}/restore	Restaura un documento desde la papelera de Drive

Recuperación y orden en Drive (Sprint 4)

Método	Ruta	Descripción
GET	/drive/recover/preview	Busca carpetas viejas de DocuKids en el Drive de la persona
POST	/drive/recover	Vincula las carpetas elegidas a hijos nuevos o existentes (hasta 20 por pedido)
POST	/drive/migrate	Ordena y renombra archivos viejos, de a 10 por llamada

Familia (Sprint 4)

Método	Ruta	Descripción
GET	/family	Devuelve la familia de la persona, o null si no tiene
POST	/family	Crea la familia; quien la crea queda como titular
POST	/family/invite	Solo el titular. Genera un código de 6 caracteres que vence en 24 horas y sirve una sola vez; reemplaza al anterior
POST	/family/join	Se une a una familia con un código
POST	/family/leave	El miembro sale de la familia
DELETE	/family/members/{user_id}	El titular quita a un miembro
DELETE	/family	El titular disuelve la familia

Turnos

Método	Ruta	Descripción
GET	/appointments/{child_id}	Lista los turnos de un hijo
POST	/appointments	Crea un turno y su evento en Google Calendar
PATCH	/appointments/{id}	Edita un turno

Método	Ruta	Descripción
DELETE	/appointments/{id}	Borra un turno

Automático

Método	Ruta	Descripción
POST	/scheduler/check	Revisa vencimientos y envía los correos. No usa token de usuario: exige la cabecera X-Cron-Secret y responde 401 si no coincide. Lo llama cron-job.org una vez al día

6. Modelo de datos

Se guardan solo metadatos; los archivos viven en el Drive de la familia. La tabla refleja `models.py` y las migraciones del backend. La pertenencia a una familia no tiene tabla propia: se guarda en `users.family_id`. El control de correos ya enviados usa `documents.last_notified_at`; no existe una tabla `alert_log`.

Tabla	Columnas
users	id (UUID de Supabase), email, full_name, google_refresh_token, google_email, family_id (vacío = sin familia), created_at
families	id, owner_id (el titular, único), invite_code, invite_expires_at, created_at
children	id, user_id, name, birth_date, avatar_url, drive_folder_id, created_at
folders	id, user_id, name, is_base, drive_folder_id, created_at. El nombre no se repite por usuario, sin distinguir mayúsculas
child_folders	child_id, folder_id, drive_folder_id, created_at. Clave compuesta (child_id, folder_id): la carpeta de cada categoría dentro de la carpeta de cada hijo en Drive

Tabla	Columnas
documents	id, child_id, name, type, expiry_date, drive_file_id, drive_link, upload_date, last_notified_at, no_expiry, folder_id, mime_type, drive_migrated, created_at
appointments	id, child_id, title, date, time, notes, calendar_event_id, created_at

Las tres carpetas de base (is_base) son Documentos, Salud y Escuela: se pueden renombrar pero no borrar.

Estado de un documento: no se guarda, se calcula en cada consulta. Los valores son: vencido (fecha pasada), proximo (vence dentro de 30 días), vigente (vence después de 30 días), sin_fecha (todavía no se cargó la fecha) y sin_vencimiento (marcado como sin vencimiento).

7. Reglas de negocio de la versión final

- **Familia:** solo el titular genera códigos, quita miembros o disuelve la familia. El código vence en 24 horas, sirve una sola vez y solo lo ve el titular mientras está vigente.
- **Datos compartidos:** un miembro trabaja sobre los datos del titular. Ve los mismos hijos, documentos, carpetas y turnos, y Drive y Calendar se usan con las credenciales del titular.
- **Google limitado al titular:** únicamente el titular conecta y administra la sincronización con Google; los miembros usan la app sin tocar esa configuración.
- **Papelera:** un documento borrado se puede restaurar durante 30 días, que es el plazo de la papelera de Drive.
- **Baja de cuenta:** la persona elige si conserva o borra sus archivos de Drive. Si los conserva, al volver a registrarse la app detecta las carpetas viejas y las vincula a los hijos en lugar de crear carpetas paralelas.
- **Correos de aviso:** se envían a partir de 15 días antes del vencimiento, una sola vez por documento. El backend reserva cada documento antes de enviar, así que si cron-job.org reintenta no hay correos repetidos.
- **Servidor gratuito:** Render duerme el servidor tras 15 minutos sin uso y el primer pedido puede tardar hasta 60 segundos. Por eso el cliente espera hasta 60 segundos, reintenta hasta 2 veces las lecturas que fallan por corte de red y nunca reintenta acciones que crean, editan o borran.

8. Estructura del frontend

Carpeta	Contenido
app/(auth)	Pantallas de login, registro y recuperación de contraseña
app/(tabs)	Las cuatro pantallas principales: index (Documentos), calendario, hijos y ajustes
components	Piezas reutilizables: selector de hijo y de avatar, tarjeta de papelera, tarjeta de recuperación de datos, ventanas de confirmación, avisos de error
context	Estado global: autenticación, hijos, familia, tema, avisos y ayuda
lib	Lógica y llamadas al backend: api, supabase, familia, papelera, carpetas, recuperarDatos, migrarDrive, errores
e2e	Prueba end-to-end con Playwright
docs	Entregas, presentación, manual de usuario e informes de pruebas

9. Pruebas y control de calidad

- **E2E con Playwright** en Chromium, Firefox y WebKit. Hoy hay una sola prueba (la app carga y tiene el título correcto).
- **Pruebas del backend:** la carpeta `test/` tiene 117 pruebas automáticas (papelera, recuperación de carpetas, migración a Drive, avatar, baja de cuenta, dueño de los datos). No usan base ni red: Drive y la base se reemplazan por dobles de prueba. Se ejecutan con pytest, que no está en `requirements.txt` y hay que instalar aparte.
- **Pruebas de integración en producción** con Chrome, Edge y Firefox: login con Google, subida a Drive, turnos en Calendar y correos.
- **Registro de bugs** con severidad, corrección y estado (carpeta docs).

10. Limitaciones conocidas

- **Privacidad:** en el nivel gratuito de Gemini, Google puede usar lo enviado para mejorar sus modelos. Se aceptó por tratarse de un proyecto académico.
- **Google en modo prueba:** la pantalla de consentimiento está en modo Externo con usuarios de prueba (hasta 100).
- **Arranque lento:** el servidor gratuito tarda en despertar después de un rato sin uso.
- **Cobertura de pruebas:** el flujo de autenticación no tiene pruebas automáticas (se postergó el mocking).
- **Pendientes del Sprint 5:** ver el compromiso del próximo período en la Entrega 6.